ConcertPro - A concert ticketing and management system

Group 105 - Laurier Lefebvre, Owen MacDonald, Johnny Weng Lin, Ayush Srivastava

Requirement Analysis:

Introduction:

This application is a concert ticketing and venue management system that stores data needed to schedule concerts, manage venues with different seat layouts and amounts, allow customers to purchase multiple tickets for venues, and allow administrators to have an overview of the concert and issue physical tickets to customers in person at a counter.

Our system will store essential data such as users, tickets, and performing artists that will enable one to schedule and manage a concert efficiently.

Scope and Special constraints/requirements:

We will limit the exploration of this concert ticketing app to the minimal requirements to have users be able to purchase tickets and basic functionality on the user and admin side, as well as allowing a modest suite of analytical and recommendation tools. It allows for the general organization of data needed to manage a concert user centric and admin centric.

Database Description:

Entities and their Attributes:

The following entities will be stored in the tables of a relational database and will be included in the ER Schema.

Users: A user is someone that is able to perform actions on the music centre interface. A user is uniquely defined by its email address and must also enter a password when signing in. However, two different users may have the same password. A user can either be a RegularUser entity or an AdminUser entity in a non-overlapping, covering ISA hierarchy.

RegularUser: We store regular users as a single entity that enables user authentication and association of actions such as online purchases, liked genres and favorite artists, that enables better organization and management of user data.

Admin: An admin user is allowed to scan/verify a regular user's ticket in person or sell a ticket to someone in person. When selling a ticket in person, the buyer does not need an account with the music centre. Instead, the admin user can simply remove a ticket from the pool of available tickets.

Tickets: Tickets are literal tickets RegularUsers can purchase. They have a price and are represented by the combination of section, seat number, and concert. Concerts a relation between a Venue and an Artist and has the attribute Date. Tickets are related to the Aggregate relation between a specific Venue and an Artist. The manner in which we store tickets in our database prevents double seat selling (same seat sold twice) and enables real time availability and pricing of events.

Artist: Artists are the entities performing at the concerts. They are represented by artistID (which is given to them by the music centre in order to identify them), have a Name and are related to one or more Genres. Artists can also be a Regular User's favourite artist.

Venue: A venue is the physical location in which the concert is held that is at most able to have one concert a night performed by a singular artist. It is represented by its full address, has a name, and the possibility of having a favourite Genre. Depending on the genre the venue chooses, specific artists of that genre would be allowed, and artists of other genres may not be allowed to perform in the specific venue. Each venue also contains a specific maximum number of seats available, i.e. its capacity.

Genre: A genre is a literal genre of music. It is represented by a Name. Regular Users can have favourite genres, Artists can be related to specific genres, and Venues can prefer certain genres. It also allows for filtering of artists and events.

Relationships:

The following relationships represent how each entity from above interacts and is interconnected in various ways.

Users ISA: There is an ISA hierarchy starting from Users going to RegularUser and Admin, where every user is **either** a **RegularUser** or an **Admin**, and this hierarchy is **covering and non-overlapping**, meaning no user can be both types and every user must be one of them.

Concert: The relationship between a Venue and an Artist. It has the attribute Date and represents a concert by an Artist at a Venue on a specific Date.This is a many-many relation, as an Artist can perform multiple times at any Venue and a Venue can host any number of Artists.

Allocated For: The relationship between ticket and the aggregate of venue and artist. The ticket is a weak entity that is identified by the specific venue and the artist that is performing in the venue. It is identified by the name of the venue and the artistID

Online Purchase: Regular users can purchase and own tickets where one regular user can be associated with 0 or more tickets and tickets are associated with at most one regular user once it is sold. One purchase can have any number of tickets associated with it. This is a

one-to-many relation as a RegularUser can buy multiple tickets, and each ticket can only be bought once.

In-person purchase: A relation between Admin and Ticket, where an Admin user sells a ticket to someone in-person and then removes it from the pool of available tickets for purchase. This is a one-to-many relation as an Admin can sell multiple tickets, and each ticket can only be sold once.

Artist Genre: An artist is related to a minimum of one genre depending on the genre of music they perform. An artist can have several genres specific to the music they play.

Venue Genre: Specific Venues can prefer to have specific Genres of music at their location. This is an optional relation, so this is a many-many relation, where one Venue can have 0+ preferred Genres, and Genres can be the favourite of any number of Venues.

Favourite Genre: RegularUsers can optionally have favourite Genres for the purpose of recommendations. This is thus a many-many relation.

Favourite Artist: RegularUsers can optionally have favourite Artists so they can be notified when they have a new Concert. This is thus also a many-many relation.

Venues Near Me: A relation between a RegularUser and a Venue that represents the Venues near a specific RegularUser. This would be a many-many relation with a participation constraint from RegularUser to the relation, so that RegularUsers can be close to one or many Venues, and Venues can have any number of RegularUsers close to it.

Scan: A relation between an Admin and a Ticket, made to represent an Admin user scanning a ticket in person. This relation is a many-one, where each ticket can only be scanned once, and an admin can scan many tickets. Not only will this relation make it possible to verify users when they go in, but we will also be able to check how many people ended up going to each event.

Functional Requirements:

The system logic supports the following tasks:

Ticket Purchase (Online): Enables a Regular User to select available seats for a specific Concert and creates a record in the online_purchase relation.

Box Office Sale(In-Person): Enables an Admin to manually remove a ticket from the pool for walk-in customers without requiring a customer account.

Entry Verification: An Admin scans a Ticket. The system checks the scan relation; if a record exists, access is denied (fraud prevention).

Preference Matching: Queries favourite_artist and favourite_genre to notify users of upcoming concerts at Venues Near Me.

Venue Management: Admins can update venue capacity or preferred genre to restrict which artists can book the space.

Application Description:

Overview

This application seeks to enable normal users such as customers to be able to purchase tickets online for a specific concert at a venue for an artist as well as enable admins to manage tickets as well as sell tickets at a counter. The information stored would also allow for venues to have a specialization for certain genres that match its location and ambiance and select for artists that match their genres.

Derived Data and Application Logic

Some values used by the application are not stored directly in the database but are derived and calculated at runtime from the existing data. This ensures consistency and reduced redundancy at real time.

Derived/Computed Values:

Seat availability per concert: The total number of available seats at real time is the total number of seats at the venue subtracted by the number of tickets already sold at that point in time.

Constraint Enforcements:

Scheduling Constraint: We will assume that a music artist can only perform in one venue at a time, and that a venue can only hold one concert per day to allow time to prepare for the concert beforehand. This allows us to focus on the customer centric components of the database.

Ticket Validity:

A ticket may only be scanned once at the venue in person in order to prevent customers from utilizing one ticket of several people. It would be enforced through the scan relationship in which the scan of a ticket would render the ticket to be invalid for further scans.

Standout Features:

Our design incorporates several sophisticated ER modeling concepts that elevate it above a standard "store" database:

1. Weak Entity Aggregation: Instead of a simple relationship, the Ticket entity is a weak entity dependent on an Aggregate of the Artist and Venue. This ensures that ticket IDs are only unique within the context of a specific event.
2. Non-Overlapping ISA Hierarchy: We implemented a covering ISA hierarchy for Users. This ensures strict security roles where an Admin cannot be a RegularUser, preventing employees from using admin privileges to purchase their own tickets.
3. Spatial Context: The Venues Near Me relationship adds a geographical layer to the data, allowing for proximity-based querying.
4. Fraud Prevention via Scan Relation: By modeling the Scan as a relationship rather than a simple boolean attribute, we maintain an audit trail of which administrator verified which ticket and at what time.

Relational Model:

Users(emailAddress, password)

RegularUser(FOREIGN KEY (emailAddress) REFERENCES Users)

Admin(FOREIGN KEY (emailAddress) REFERENCES Users)

Genre(genreName)

favourite_genre(FOREIGN KEY (emailAddress) REFERENCES RegularUser, FOREIGN KEY (genreName) REFERENCES Genre)

Artist(artistId, name)

favourite_artist(FOREIGN KEY (emailAddress) REFERENCES RegularUser, FOREIGN KEY (artistId) REFERENCES Artist)

artist_genre(FOREIGN KEY (genreName) REFERENCES Genre, FOREIGN KEY (artistId) REFERENCES Artist)

Venue(address, venueName, capacity)

venues_near_me(FOREIGN KEY (emailAddress) REFERENCES RegularUser, FOREIGN KEY (address) REFERENCES Venue)

venue_genre(FOREIGN KEY (genreName) REFERENCES Genre, FOREIGN KEY (address) REFERENCES Venue)

concert(FOREIGN KEY (artistId) REFERENCES Artist, FOREIGN KEY (address) REFERENCES Venue, date)

Tickets(FOREIGN KEY (artistId) REFERENCES Artist, FOREIGN KEY (address) REFERENCES Venue, section, seatNumber, price)

online_purchase(FOREIGN KEY (artistId, address, section, seatNumber) REFERENCES Tickets, FOREIGN KEY (emailAddress) REFERENCES RegularUser)

in-person_purchase(FOREIGN KEY (artistId, address, section, seatNumber) REFERENCES Tickets, FOREIGN KEY (emailAddress) REFERENCES Admin)

scan(FOREIGN KEY (artistId, address, section, seatNumber) REFERENCES Tickets, FOREIGN KEY (emailAddress) REFERENCES Admin)

Notes:
Unable to show participation constraints for artist_genre and venues_near_me, shown as many-many relationships instead.

Inspiration:

These are some of the sites that we used as inspiration:
https://www.ticketmaster.ca
https://www.tickpick.com

Collaboration:

We first had a meeting to decide the topic of the project. During that same meeting we all came up with the ideas for most of the entity sets and relations, along with their attributes. We then delegated tasks: Laurier Lefebvre worked on finishing and formalizing the entity sets and relations, and did the ER diagram. Owen MacDonald worked on finishing and formalizing the entity sets and relations, and did the Relational Model. Johnny Weng Lin worked on the database description focusing on the entities and some of the relations. Mainly on defining key terms and how they relate to each other. Also worked on the application description. Ayush Srivastava worked on the overall application description and refined the final report.